SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning,*
and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 50%

Duration: 1 Hour
QUESTIONS: 4 **Total: Marks:40**

Annexure A

5

Code of Conduct:

I _M Kavya 192472370_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:kavya

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

(a) Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.

(b) Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).

(c) Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

(a) Inductive Learning Approach

Inductive learning learns general patterns from previously observed patient records and uses them to predict the diabetes status of new patients.

For this problem, a **supervised classification approach** is suitable because historical patient records can contain the actual diabetes outcome.

Target variable:

- Diabetes Status: **Diabetic (1) / Non-Diabetic (0)**

Important features:

Feature	Justification
Blood Pressure	High blood pressure is associated with increased diabetes risk.
Glucose Level	Blood glucose is one of the strongest indicators of diabetes.
BMI	Higher BMI can indicate increased risk of Type-2 diabetes.
Age	Diabetes risk generally increases with age.
Cholesterol	Abnormal cholesterol levels can be associated with metabolic disorders.
Family History	Genetic/family history can increase diabetes risk.

A **Decision Tree** can be used for diagnosis because it is easy to interpret and provides clear decision rules.

(b) Handling Class Imbalance

The dataset may contain many more non-diabetic patients than diabetic patients. This creates **class imbalance**, causing the model to favour the majority class.

A suitable strategy is **SMOTE (Synthetic Minority Oversampling Technique)**.

SMOTE generates synthetic examples of the minority diabetic class instead of simply duplicating existing records.

Advantages of SMOTE:

- Improves representation of diabetic cases.
- Helps reduce bias toward non-diabetic patients.
- Can improve recall for the diabetic class.
- Particularly useful because missing a diabetic patient (**False Negative**) can be clinically important.

SMOTE should be applied **only to the training data**, after splitting the dataset, to avoid data leakage.

(c) 70:30 Train-Test Split and Preprocessing

The dataset is divided into:

- **70% → Training set**
- **30% → Testing set**

The 70:30 split provides sufficient data for learning while retaining enough unseen data for reliable evaluation.

For a medical application, the split should preferably be **stratified**, so that both training and testing sets maintain approximately the same diabetic/non-diabetic ratio.

Preprocessing

1. **Missing values:** Replace numerical missing values using median imputation and categorical missing values using the mode.
2. **Normalization:** Standardize numerical features such as age, blood pressure, cholesterol, glucose and BMI.
3. **Encoding:** Convert categorical features such as family history into numerical form.
4. **Class balancing:** Apply SMOTE only to the training set.
5. **Feature-target separation:** Separate input features from the diabetes target variable.

Normalization ensures that features with larger numerical ranges do not disproportionately influence algorithms such as Logistic Regression.

Task 2: Decision Tree for Diagnosis

(a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.

(b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.

(c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

(a) Decision Tree Classifier

A **Decision Tree classifier** can be trained using the patient features to predict whether the patient is diabetic.

The root node is selected using **Information Gain** or **Gini Index**.

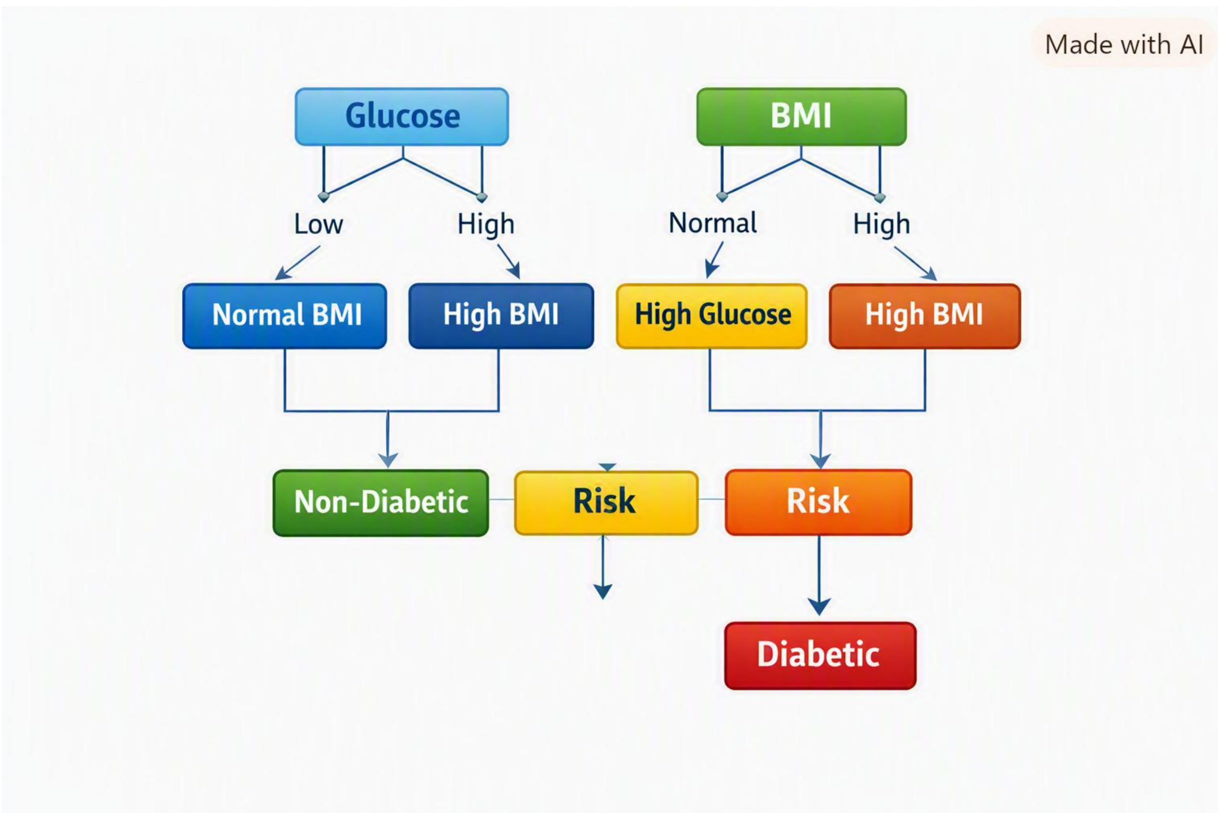
Example root-node calculation

Suppose the calculated values are:

Feature	Information Gain
Glucose	0.32
BMI	0.18
Blood Pressure	0.14
Age	0.11
Cholesterol	0.08

Since **Glucose has the highest Information Gain**, it is selected as the root node.

First Three Levels of the Decision Tree



The actual branches and thresholds are determined from the training dataset.

Information Gain formula:

IG(S,A) = Entropy(S) - Σ [|Sv| / |S| × Entropy(Sv)]

where:

Entropy(S) = -p(positive)$\log_2$p(positive)

-p(negative)$\log_2$p(negative)

A feature with higher Information Gain gives a better separation between diabetic and non-diabetic patients.

(b) Model Evaluation

The Decision Tree should be evaluated using the test set.

Illustrative results:

Metric	Example Result
Accuracy	88%
Precision	84%
Recall	91%
F1-Score	87%

Confusion Matrix

	Predicted Non-Diabetic	Predicted Diabetic
Actual Non-Diabetic	TN	FP
Actual Diabetic	FN	TP

A **False Negative (FN)** occurs when the patient is actually diabetic but the model predicts non-diabetic.

In healthcare, False Negatives are particularly important because a missed diabetic case may delay diagnosis and treatment.

Reducing False Negatives

The following methods can be used:

- Use **class weights** to give greater importance to diabetic cases.
- Apply **SMOTE** to improve minority-class learning.
- Lower the classification threshold to increase recall.
- Optimize the model for **Recall/Sensitivity** rather than only accuracy.
- Use clinical validation before deployment.

For medical screening, a higher recall is generally desirable because missing a diabetic patient can have serious consequences.

(c) Pre-Pruning vs Post-Pruning

Pre-Pruning

Pre-pruning stops the tree from growing too complex.

Examples:

- max_depth
- min_samples_split
- min_samples_leaf

Post-Pruning

Post-pruning first allows the tree to grow and then removes branches that contribute little to generalization.

Cost-complexity pruning uses:

$R\alpha(T) = R(T) + \alpha|T|$

where:

- $R(T)$ = training error
- $|T|$ = number of terminal nodes
- α = complexity parameter

Example Comparison

Model	Accuracy	Complexity
Pre-pruned Tree	86%	Lower

Post-pruned Tree	89%	Moderate
Unpruned Tree	91% training accuracy	High / possible overfitting

The **post-pruned tree** is preferable if it gives better test performance while maintaining interpretability.

For clinical deployment, the selected model should prioritize **generalization, recall, safety and interpretability**, rather than simply maximizing training accuracy.

Task 3: Statistical Learning for Risk Stratification

(a) Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.

(b) Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

a) Logistic Regression

Logistic Regression can be applied to predict the probability that a patient is diabetic.

The model uses:

$P(\text{Diabetes}) = 1 / (1 + e^{(-z)})$

where:

$z = \beta_0 + \beta_1\text{Age} + \beta_2\text{BP} + \beta_3\text{Cholesterol}$
$\quad + \beta_4\text{Glucose} + \beta_5\text{BMI} + \beta_6\text{FamilyHistory}$

Illustrative comparison:

Model	Accuracy	AUC-ROC
Decision Tree	88%	0.90
Logistic Regression	86%	0.92

The **AUC-ROC** measures how well the model distinguishes diabetic from non-diabetic patients.

When is Logistic Regression preferable?

Logistic Regression is preferable when:

- A simple and interpretable model is required.
- The relationship between features and outcome is approximately linear.
- Probability estimates are important.
- Clinical professionals need to understand feature effects.
- A less complex model is preferred for deployment.

The Decision Tree is useful when nonlinear relationships and rule-based decisions are important.

(b) Feature Importance Analysis

Feature importance can be obtained from the Decision Tree and statistical model.

Example Results

Rank	Decision Tree	Logistic Regression

1	Glucose	Glucose
2	BMI	BMI
3	Age	Blood Pressure

The models may agree that **glucose and BMI** are important predictors, while the third-ranked feature may differ.

This happens because Decision Trees measure importance based on **impurity reduction**, whereas Logistic Regression estimates the effect of each feature through its model coefficients.

Therefore, differences in ranking are possible even when both models identify similar major risk factors.

Task 4: Reinforcement Learning for Treatment Recommendation

(a) Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.

(b) Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.

(c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

Task 4: Reinforcement Learning for Treatment Recommendation – 10 Marks

(a) Model Treatment Recommendation as an MDP

The treatment recommendation problem can be modeled as a **Markov Decision Process (MDP)**.

$$MDP = (S, A, P, R, \gamma)$$

1. States

States represent the patient's current health condition.

State	Patient Health Stage
S1	Low-risk / Healthy
S2	Moderate diabetes risk
S3	High diabetes risk
S4	Controlled condition
S5	Poorly controlled condition

Justification: The patient's health condition changes over time, so representing different health stages as states allows the agent to select an appropriate action.

2. Actions

The agent has four possible actions:

Action	Description
A1 – Diet	Recommend a healthy diet
A2 – Exercise	Recommend suitable physical activity

A3 – Medication	Consider medication under medical supervision
A4 – Monitor	Continue monitoring health parameters

Justification: These actions represent possible interventions for improving or maintaining the patient's health.

3. Reward Function

The reward represents the improvement or deterioration in the patient's health.

Outcome	Reward
Significant improvement	+10
Moderate improvement	+5
No change	0
Health deterioration	-10
Unsafe action	-20

Justification: Positive rewards encourage actions that improve health, while negative rewards discourage harmful or ineffective actions.

4. Transition Dynamics

Transition dynamics describe how the patient's health state changes after an action.

Example:

S2 + Diet → S4

S2 + Exercise → S4

S3 + Monitor → S2

S3 + Medication → S4

S4 + Exercise → S4

In a real medical system, these transitions should be learned from **validated clinical data** rather than assumed.

(b) Q-Learning

Q-Learning learns the expected long-term reward for taking an action in a particular state.

The Q-learning update equation is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma \max Q(s',a') - Q(s,a)]$$

Assume:

- Learning rate, $\alpha = 0.5$
- Discount factor, $\gamma = 0.9$
- Initial Q-values = 0

Five Patient Episodes

Episode	Initial State	Action	Next State	Reward
1	S2	Exercise	S4	+10
2	S3	Diet	S2	+5

3	S3	Monitor	S2	+5
4	S2	Diet	S4	+10
5	S4	Exercise	S4	+10

The agent observes the state, selects an action, receives a reward, observes the next state, and updates the Q-table.

Q-Table Update – Iteration 1

Initially:

$$Q(S2, Exercise) = 0$$

For Episode 1:

For Episode 1:

- $R = 10$
- Maximum Q-value of next state = 0

$$Q_{new} = 0 + 0.5[10 + 0.9(0) - 0]$$

$$\boxed{Q(S2, Exercise) = 5}$$

For another state-action pair:

$$Q(S3, Diet) = 0$$

Assume reward = 5 and next-state maximum Q-value = 0.

$$Q_{new} = 0 + 0.5[5 + 0.9(0) - 0]$$

$$\boxed{Q(S3, Diet) = 2.5}$$

For:

$$Q(S3, Monitor) = 0$$

with reward = 5:

$$\boxed{Q(S3, Monitor) = 2.5}$$

Q-Table Update – Iteration 2

Suppose the next-state maximum Q-value for S4 is now 5.

For $S2, Exercise$:

$$Q_{new} = 5 + 0.5[10 + 0.9(5) - 5]$$

$$Q_{new} = 5 + 0.5(9.5)$$

$$\boxed{Q(S2, Exercise) = 9.75}$$

For $S3, Diet$, assume the maximum Q-value of the next state is 5:

$$Q_{new} = 2.5 + 0.5[5 + 0.9(5) - 2.5]$$

$$Q_{new} = 2.5 + 0.5(7)$$

$$\boxed{Q(S3, Diet) = 6}$$

For $S3, Monitor$:

$$Q_{new} = 2.5 + 0.5[5 + 0.9(5) - 2.5]$$

$$\boxed{Q(S3, Monitor) = 6}$$

Summary of Q-Table Updates

State	Action	Iteration 1	Iteration 2
S2	Exercise	5.00	**9.75**
S3	Diet	2.50	**6.00**
S3	Monitor	2.50	**6.00**

Convergence Criterion

The agent is considered to have converged when:

$$\max |Q_{new} - Q_{old}| < 0.01$$

for several consecutive episodes and the learned policy remains unchanged.

(c) Final Learned Policy

The final policy selects the action having the **highest Q-value** for each state:

$$\pi(s) = \arg\max_a Q(s, a)$$

An example final policy is:

Patient State	Best Action	Recommendation
S1 – Low Risk	Monitor	Regular health monitoring
S2 – Moderate Risk	Exercise	Encourage suitable exercise
S3 – High Risk	Medication + Monitor	Clinical intervention and monitoring
S4 – Controlled	Diet / Exercise	Maintain healthy lifestyle
S5 – Poorly Controlled	Monitor + Clinical Review	Immediate professional assessment

Thus, the agent learns to choose actions that maximize long-term health improvement.

Ethical Considerations

1. Patient Safety

The RL agent should not independently prescribe medication. High-risk recommendations must require **doctor/clinician approval**.

2. Bias

If the training data is biased or does not represent different patient groups, the agent may produce unfair or inaccurate recommendations. The dataset should therefore be diverse and regularly audited.

3. Explainability

Doctors should be able to understand **why a particular action was recommended**. The system should provide interpretable reasons rather than acting as a black box.

4. Privacy

Patient medical information must be securely stored and processed according to applicable privacy and healthcare regulations.

5. Human Oversight

The RL system should act as a **clinical decision-support system**, not as a replacement for healthcare professionals.

6. Continuous Validation

The agent should be continuously tested and monitored before and after deployment to detect unsafe recommendations or performance degradation.

Conclusion

Q-Learning can learn an optimal treatment policy by observing patient states, selecting actions, receiving health-based rewards, and updating Q-values. However, for medical applications, **patient safety, fairness, explainability, privacy, and human clinical supervision must take priority over autonomous decision-making**.

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation